

ITEM HIVE

Solution Architecture

Table of Contents

1. Executive Context & System Purpose 2
2. High-Level System Architecture Diagram (3-Tier Web Architecture) 3
3. Tech Stack & Layer Responsibilities 4
   - Tier 1: Presentation Layer (Frontend) 4
   - Tier 2: Application Layer (Backend & Business Logic) 4
   - Tier 3: Data Tier (Database Schema) 4
4. Key Technical Workflows (Web-Based) 5
   - Web Order Customization & Pre-Order Flow 5
5. Non-Functional Requirements (NFRs) 5

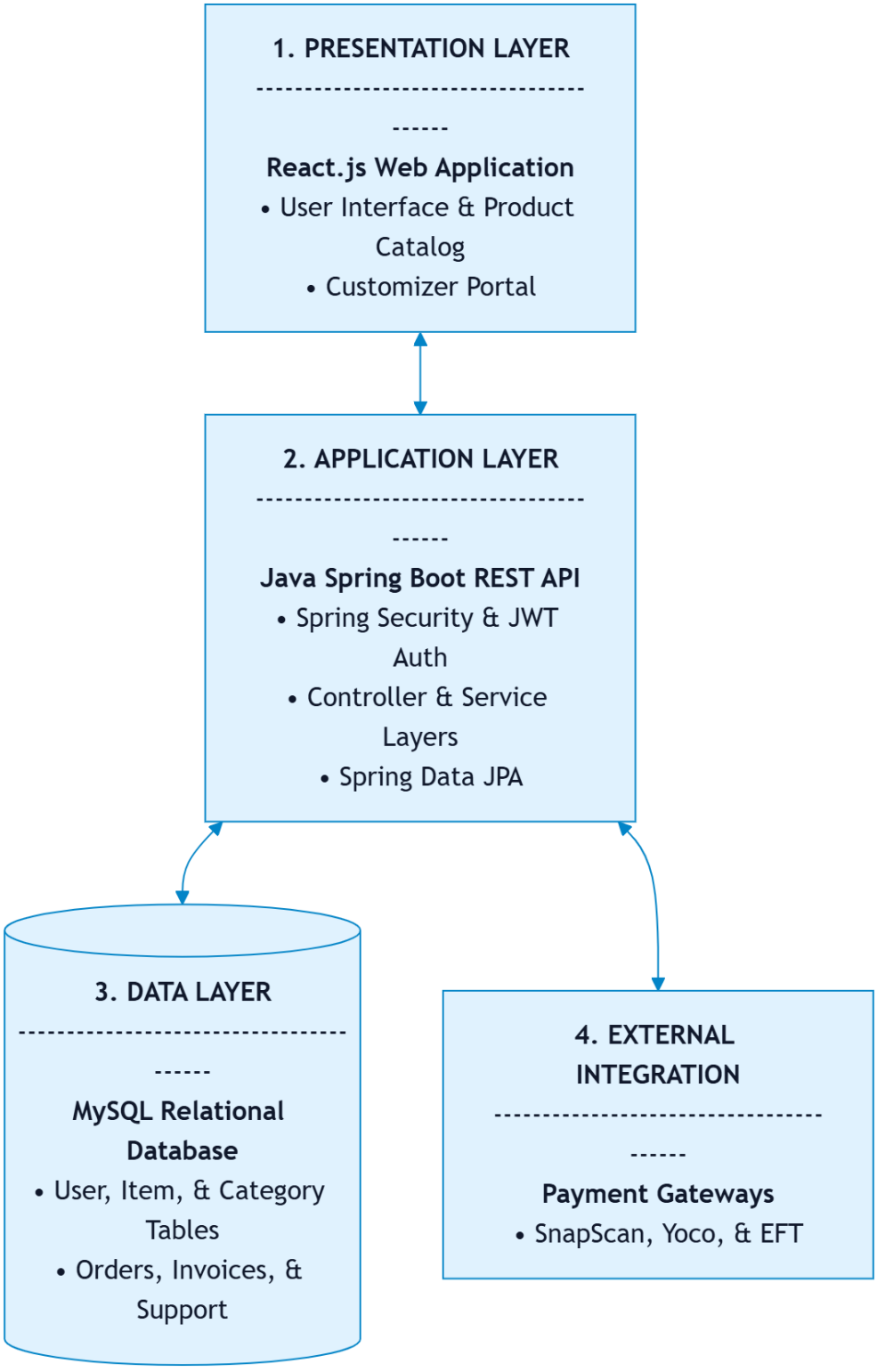
1. Executive Context & System Purpose

The primary goal of Item Hive is to provide a reliable, full-stack micro-e-commerce web platform for CPUT District Six ICT students. The system addresses financial constraints and campus identity by offering affordable, customizable apparel.

The web architecture supports:

- Browsing categorized apparel (unisex, crop tops, oversized cuts) via web browsers.
- Interactive web customization (selecting programming language artwork like Java/Python/SQL and custom sleeve/back name prints).
- Pre-orders and flexible payment integration (Yoco, SnapScan and EFT).
- Web-based Admin portal for inventory management, order processing, and support ticket resolution.

2. High-Level System Architecture Diagram



3. Tech Stack & Layer Responsibilities

Tier 1: Presentation Layer (Frontend)

Technology: React.js (JavaScript/JSX, CSS Modules / Tailwind, Axios for HTTP requests).

Responsibilities:

- Renders web pages (Catalog, Item Customizer, Shopping Cart, Pre-Order Checkout, Admin Dashboard).
- Captures student inputs for custom programming logos (Java, Python, C#, SQL) and custom name text.
- Handles client-side route protection (Admin vs. Student access) using stored JWT tokens.
- Displays embedded payment modals/QR links for Yoco and SnapScan checkout.

Tier 2: Application Layer (Backend & Business Logic)

Technology: Java Spring Boot (Spring MVC, Spring Security, Spring Data JPA).

Responsibilities:

- Spring Security & JWT: Validates user login, hashes passwords (via BCryptPasswordEncoder), and secures endpoints with stateless JWT filters.
- REST Controllers (@RestController): Exposes web endpoints for user auth, product catalogues, shopping carts, pre-orders and support tickets.
- Service Layer (@Service): Houses business logic for processing pre-orders, applying customized attributes and calculating layby deposit balances.
- Payment Webhook Listener: Accepts payment completion webhooks from Yoco/SnapScan to automatically update order statuses in MySQL.

Tier 3: Data Tier (Database Schema)

Technology: MySQL Relational Database (accessed via Spring Data JPA / Hibernate ORM).

Core Entities & Relationships:

- User: `user_id` (PK), `student_number` and `password_hash`(ROLE_STUDENT).
- Category: `category_id` (PK), `category_name` (Oversized, Crop, Unisex).
- Item: `item_id` (PK), `category_id` (FK), `title`, `base_price`, `stock_quantity`.
- Cart & CartItem: Tracks active web session selections, sizes, and custom text inputs before pre-order submission.

- Order & OrderItem: `order_id` (PK), `user_id` (FK), `status` (PENDING, PRINTING, READY), `custom_prog_language`, `custom_name_text`.
- Invoice: `invoice_id` (PK), `order_id` (FK), `payment_method`, `amount_paid`, `balance_remaining`.
- SupportTicket: `ticket_id` (PK), `user_id` (FK), `subject`, `message`, `status`.

4. Key Technical Workflows (Web-Based)

Web Order Customization & Pre-Order Flow

1. **User Action:** Student selects an oversized t-shirt on the React website, picks the "Java" logo, types of custom name text and clicks "Add to Cart".
2. **Frontend Request:** React issues a POST /api/cart/items payload via Axios containing the customization parameters.
3. **Backend Action:** Spring Boot validates the user session JWT, creates/updates the CartItem entity and saves it to MySQL via Spring Data JPA.
4. **Checkout & Payment:**
    - User navigates to checkout and chooses SnapScan, Yoco or EFT
    - Spring Boot generates an Order in state PENDING.
5. **Webhook Callback:**
    - Upon payment, Yoco/SnapScan pings /api/payments/webhook.
    - The webhook service updates the order status in MySQL to PRINTING.

5. Non-Functional Requirements (NFRs)

Metric	Architectural Strategy
Security	Passwords hashed using Spring Security BCrypt; API routes protected via JWT Bearer tokens; CORS enabled exclusively for the React frontend domain.
Data Integrity	MySQL ACID compliance ensures transaction safety during simultaneous pre-order drops.
Maintainability	Clean Spring Boot multi-layer package structure (controller, service, repository, model, dto.
Web Performance	Spring Boot JPA query optimization and React component caching for fast browser load speeds.